Library App

Backend API Documentation

Complete endpoint reference · Request & response schemas · Database design

Generated: June 2026

20	4	4	JWT
Endpoints	Modules	DB Tables	Auth

1. API Overview

Base URL

```
https:///api
```

Authentication

Protected endpoints require a Bearer JWT token in the Authorization header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

User Roles

Role	Access	Notes
Admin	Full access	Can delete resources; manage all users
Librarian	Manage books, categories, transactions	Cannot perform Admin-only destructive operations
Member	Browse, borrow, own transactions	Cannot access other users' data
Public	Browse books, register, login	No token required

HTTP Status Codes

Code	Meaning
`200 OK`	Request succeeded
`201 Created`	Resource created successfully
`400 Bad Request`	Validation failed or invalid input
`401 Unauthorized`	Missing or invalid JWT token
`403 Forbidden`	Authenticated but insufficient permissions
`404 Not Found`	Requested resource does not exist

2. API Endpoints

[AUTH] Authentication

POST `/api/Auth/register` — Public

Registers a new user. Defaults to Admin role via seed logic.

Request

```
{
"name": "Jane Doe",
"email": "jane@example.com",
"password": "Password123"
}
```

Response (200)

```
{
"message": "User registered successfully."
}
```

Status codes: 200 OK 400 Bad Request (email exists or validation fails)

POST `/api/Auth/login` — Public

Validates credentials and active status; returns a JWT token.

Request

```
{
"email": "jane@example.com",
"password": "Password123"
}
```

Response (200)

```
{
"message": "Login successful.",
"token": "eyJhbGciOi...",
"user": { "userId": 1, "name": "Jane Doe",
"email": "jane@example.com", "role": "Admin" }
}
```

Status codes: 200 OK 401 Unauthorized (invalid credentials or account deactivated)

GET `/api/Auth` — Public

Returns a list of all registered users.

Request

No body required.

Response (200)

```
[{ "userId": 1, "name": "Jane Doe", "email":
"jane@example.com",
"role": "Admin", "isActive": true, "createdAt":
"2026-06-03T17:34:52Z" }]
```

Status codes: 200 OK

GET `/api/Auth/{id}` — Public

Returns a specific user's details including active and overdue transactions.

Request

```
Path param: id (integer — User ID)
```

Response (200)

```
{
"userId": 1, "name": "Jane Doe",
"activeTransactions": [
{ "transactionId": 12, "bookTitle": "Clean Code",
"borrowDate": "2026-06-01T10:00:00Z", "status":
"Active" }
]
}
```

```
Status codes: 200 OK 404 Not Found
```

PATCH `/api/Auth/{id}/toggle-status` — Public

Toggles user's active status (suspend or reactivate).

Request

```
Path param: id (integer — User ID)
```

Response (200)

```
{
"message": "User account has been suspended."
}
```

```
Status codes: 200 OK 404 Not Found
```

[CAT] Categories

GET `/api/Categories` — Bearer Token — All Roles

Returns all book categories, ordered alphabetically.

Request

No body required.

Response (200)

```
[{ "categoryId": 1, "name": "Fiction",
"description": "Fictional literature and novels"
}]
```

```
Status codes: 200 OK 401 Unauthorized
```

GET `/api/Categories/{id}` — Bearer Token — All Roles

Returns details of a specific category.

Request

```
Path param: id (integer — Category ID)
```

Response (200)

```
{ "categoryId": 1, "name": "Fiction",
"description": "Fictional literature and novels"
}
```

```
Status codes: 200 OK 401 Unauthorized 404 Not Found
```

POST `/api/Categories` — Bearer Token — Admin, Librarian

Creates a new book category.

Request

```
{
"name": "Science Fiction",
"description": "Sci-Fi novels"
}
```

Response (200)

```
{ "categoryId": 4, "name": "Science Fiction",
"description": "Sci-Fi novels" }
```

Status codes: 201 Created 400 Bad Request 401 Unauthorized 403 Forbidden

PUT `/api/Categories/{id}` — Bearer Token — Admin, Librarian

Updates name and description of an existing category.

Request

```
Path param: id (integer)
Body: { "name": "Sci-Fi", "description": "Science
fiction novels" }
```

Response (200)

```
{ "message": "Category updated successfully." }
```

Status codes: 200 OK 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found

DEL `/api/Categories/{id}` — Bearer Token — Admin

Deletes a category. Blocked if any books are assigned to it.

Request

```
Path param: id (integer — Category ID)
```

Response (200)

```
{ "message": "Category deleted successfully." }
```

Status codes: 200 OK 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found

[TXN] Transactions

POST `/api/Transactions/borrow` — Bearer Token — All Roles

Borrows a book copy; auto-decrements available copies.

Request

```
{
"bookId": 3,
"userId": 1
}
```

Response (200)

```
{
"message": "Book borrowed successfully!",
"transactionId": 15,
"dueDate": "2026-06-17T17:34:52Z"
}
```

Status codes: 200 OK 400 Bad Request (out of stock/inactive user) 401 Unauthorized 404 Not Found

PATCH /api/Transactions/{id}/status

Bearer Token — Admin, Librarian

Updates transaction status. Values: 0=Active 1=Returned 2=Overdue 3=Lost. Auto-increments copies on Returned.

Request

```
Path param: id (integer — Transaction ID)
Body: { "status": 1 }
```

Response (200)

```
{ "message": "Transaction status updated to Returned." }
```

```
Status codes: 200 OK 401 Unauthorized 403 Forbidden 404 Not Found
```

GET /api/Transactions

Bearer Token — Admin, Librarian

Returns full transaction history across all users, ordered by borrow date descending.

Request

No body required.

Response (200)

```
[{
"transactionId": 15, "userId": 1, "userName": "Jane Doe",
"bookId": 3, "bookTitle": "Clean Code",
"borrowDate": "2026-06-03T17:34:52Z",
"dueDate": "2026-06-17T17:34:52Z",
"returnDate": null, "transactionStatus": "Active"
}]
```

```
Status codes: 200 OK 401 Unauthorized 403 Forbidden
```

GET /api/Transactions/user/{userId}

Bearer Token — Owner or Admin

Returns transaction history for a specific user. Members cannot query other users.

Request

```
Path param: userId (integer — User ID)
```

Response (200)

```
[{ "transactionId": 15, "bookTitle": "Clean Code",
"borrowDate": "2026-06-03T17:34:52Z",
"dueDate": "2026-06-17T17:34:52Z",
"returnDate": null, "transactionStatus": "Active"
}]
```

```
Status codes: 200 OK 401 Unauthorized 403 Forbidden
```

[BOOK] Books

GET **/api/Books** Public

Returns paginated book list. PageSize capped at 50.

Request

```
Query params: PageNumber (default 1), PageSize (default 10, max 50),
CategoryId (optional), IsActive (optional), OnlyAvailable (optional)
```

Response (200)

```
{
"items": [{ "bookId": 3, "title": "Clean Code",
"author": "Robert C. Martin", "availableCopies": 4,
"totalCopies": 5, "categoryName": "Science & Tech" }],
"totalCount": 1, "pageNumber": 1, "pageSize": 10
}
```

Status codes: 200 OK

GET **/api/Books/search** Public

Autocomplete search. Matches Title or Author; results capped at 13.

Request

```
Query param: term (string — search term)
```

Response (200)

```
[{ "bookId": 3, "title": "Clean Code",
"author": "Robert C. Martin", "availableCopies": 4 }]
```

Status codes: 200 OK

GET **/api/Books/{id}** Public

Returns details for a single book.

Request

```
Path param: id (integer — Book ID)
```

Response (200)

```
{ "bookId": 3, "title": "Clean Code", "author": "Robert C. Martin",
"totalCopies": 5, "availableCopies": 4,
"isActive": true, "categoryName": "Science & Tech" }
```

Status codes: 200 OK 404 Not Found

POST **/api/Books** Bearer Token — Admin, Librarian

Adds a new book. AvailableCopies is set equal to TotalCopies on creation.

Request

```
{
"title": "Refactoring", "author": "Martin Fowler",
"description": "Improving the Design of Existing Code",
"totalCopies": 3, "categoryId": 3, "isActive": true
}
```

Response (200)

```
{ "bookId": 4, "title": "Refactoring",
"availableCopies": 3, "categoryName": "Science & Tech" }
```

Status codes: 201 Created 400 Bad Request 401 Unauthorized 403 Forbidden

PUT **/api/Books/{id}** Bearer Token — Admin, Librarian

Updates book details. TotalCopies cannot be reduced below currently borrowed count.

Request

```
Path param: id (integer)
Body: { "title": "Refactoring (2nd Ed.)",
"totalCopies": 4, "categoryId": 3, "isActive":
true }
```

Response (200)

```
{ "message": "Book updated successfully." }
```

```
Status codes: 200 OK 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found
```

DEL **/api/Books/{id}** Bearer Token — Admin

Deletes a book. Blocked if there are Active or Overdue transactions.

Request

```
Path param: id (integer — Book ID)
```

Response (200)

```
{ "message": "Book deleted successfully." }
```

```
Status codes: 200 OK 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found
```

3. Database Schema

Relational SQL database with four tables. Foreign keys enforce referential integrity. All timestamps in UTC.

Users

Stores all user accounts (members and staff).

Column	Data Type	Constraints / Notes
UserId	`INT`	PK Auto-increment
Name	`NVARCHAR(100)`	Required
Email	`NVARCHAR(255)`	Required Unique Email format
Password	`NVARCHAR(MAX)`	Required Stored hashed
Role	`INT`	0=Admin 1=Librarian 2=Member Default: Member
IsActive	`BIT`	Default: true
CreatedAt	`DATETIME`	Default: UTC Now

Categories

Classification labels assigned to books.

Column	Data Type	Constraints / Notes
CategoryId	`INT`	PK Auto-increment
Name	`NVARCHAR(100)`	Required Max 100 chars
Description	`NVARCHAR(MAX)`	Optional

Books

Master catalogue of all books held by the library.

Column	Data Type	Constraints / Notes
BookId	`INT`	PK Auto-increment
Title	`NVARCHAR(255)`	Required
Author	`NVARCHAR(255)`	Required
Description	`NVARCHAR(MAX)`	Optional
ImageUrl	`NVARCHAR(MAX)`	Optional
TotalCopies	`INT`	Required
AvailableCopies	`INT`	Required Auto-managed on borrow/return
IsActive	`BIT`	Default: true
CategoryId	`INT`	FK -> Categories.CategoryId

Transactions

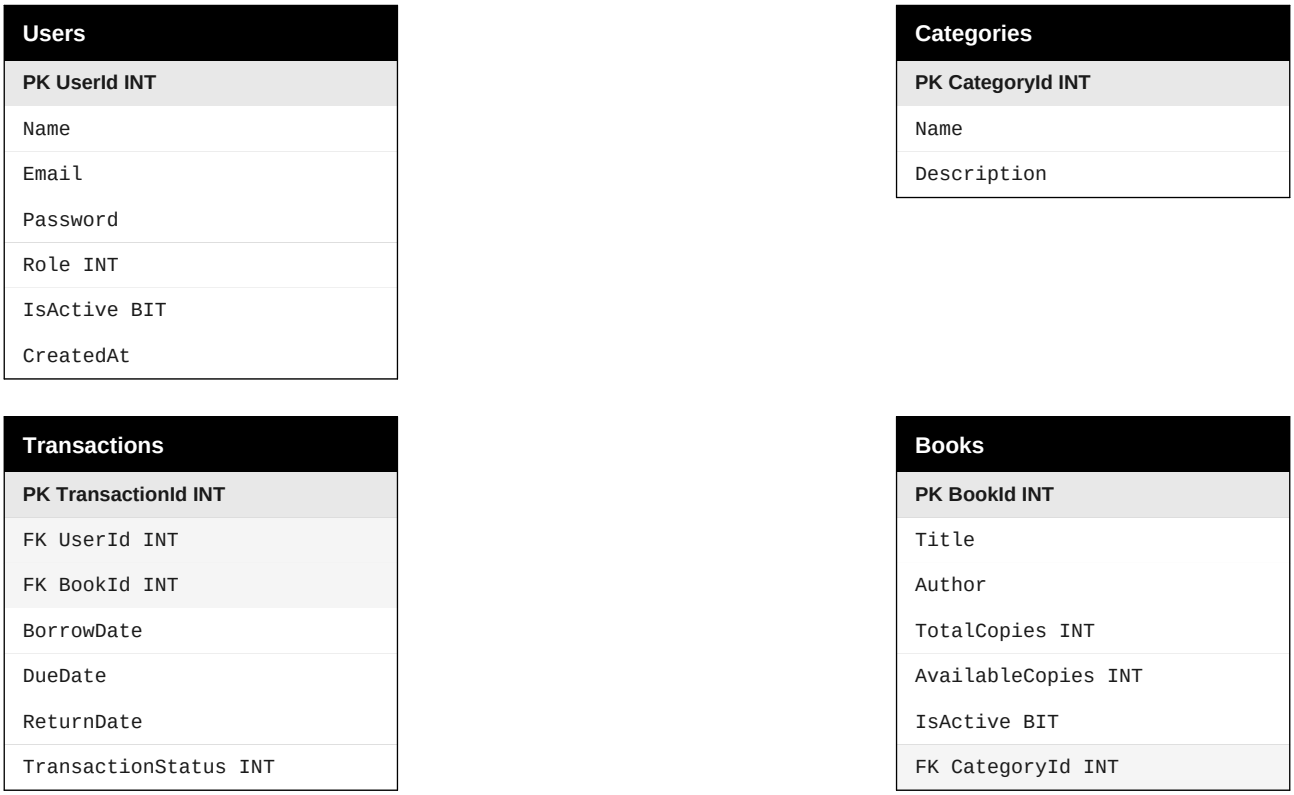
Records every borrow event and its lifecycle.

Column	Data Type	Constraints / Notes

TransactionId	`INT`	PK Auto-increment
UserId	`INT`	FK -> Users.UserId
BookId	`INT`	FK -> Books.BookId
BorrowDate	`DATETIME`	Required Default: UTC Now
DueDate	`DATETIME`	Required BorrowDate + 14 days
ReturnDate	`DATETIME`	Nullable Set on actual return
TransactionStatus	`INT`	0=Active 1=Returned 2=Overdue 3=Lost Default: Active

4. Database Design

Entity-Relationship Overview



Relationships

Relationship	Type	Description
Categories → Books	One-to-Many	One category holds many books; each book belongs to one category.
Books → Transactions	One-to-Many	One book can have many transaction records over its lifetime.
Users → Transactions	One-to-Many	One user can have many borrow transactions.

Business Rules

1	AvailableCopies is auto-decremented on borrow and incremented on return.
2	TotalCopies cannot be reduced below the count of currently borrowed copies.
3	A Category cannot be deleted if books are assigned to it.
4	A Book cannot be deleted if it has Active or Overdue transactions.
5	A Member can only view their own transactions.
6	Deactivated users cannot borrow books or log in.